

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»
(Университет ИТМО)

Факультет **Прикладной информатики**
Образовательная программа **Мобильные и сетевые технологии**
Направление подготовки(специальность) **09.03.03 Прикладная информатика**

ЛАБОРАТОРНАЯ РАБОТА №6

По дисциплине «Проектирование и реализация баз данных»

Тема: Введение в СУБД MongoDB. Установка MongoDB. Начало работы с БД. Работа с БД в СУБД MongoDB.

Выполнил	Мезенцев Б.Г.; К 3239.
Проверил	Говорова М.М.
Дата	16.05.2026

Санкт-Петербург 2026

1 Цель работы

- 1) Овладеть практическими навыками установки СУБД MongoDB.
- 2) Овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

2 Практические задания

Практическое задание для ЛР 6.1:

- Установите MongoDB для обеих типов систем (32/64 бита).
- Проверьте работоспособность системы запуском клиента mongo.
- Выполните методы:
 - a) `db.help()`
 - b) `db.help`
 - c) `db.stats()`
- Создайте БД learn.
- Получите список доступных БД.
- Создайте коллекцию unicorns, вставив в нее документ `{name: 'Aurora', gender: 'f', weight: 450}`.
- Просмотрите список текущих коллекций.
- Переименуйте коллекцию unicorns.
- Просмотрите статистику коллекции.
- Удалите коллекцию.
- Удалите БД learn.

Практическое задание для ЛР 6.2: см. пункт 5 “Выполнение лабораторной работы №6.2”

4 Выполнение лабораторной работы №6.1

4.1 Установка СУБД MongoDB

Для выполнения лабораторной работы использовался пакетный менеджер Homebrew. С его помощью была установлена утилита MongoDB Atlas CLI, предназначенная для управления облачными кластерами MongoDB Atlas.

Установка выполнялась командой:

```
brew install mongodb-atlas
```

```
6ogdanmezentsev@MacBook-Air ~ % brew install mongodb-atlas
[==> Fetching downloads for: mongodb-atlas-cli
✓ Bottle Manifest mongodb-atlas-cli (1.54.0)      Downloaded 35.6KB/ 35.6KB
✓ Bottle Manifest fmt (12.1.0)                   Downloaded 7.3KB/ 7.3KB
✓ Bottle Manifest ada-url (3.4.4)                 Downloaded 8.7KB/ 8.7KB
✓ Bottle Manifest brotli (1.2.0)                  Downloaded 8.0KB/ 8.0KB
✓ Bottle Manifest c-ares (1.34.6)                 Downloaded 7.5KB/ 7.5KB
✓ Bottle Manifest hdrhistogram_c (0.11.9)         Downloaded 7.8KB/ 7.8KB
✓ Bottle Manifest icu4c@78 (78.3)                 Downloaded 9.7KB/ 9.7KB
✓ Bottle Manifest libnghttp2 (1.69.0)             Downloaded 7.3KB/ 7.3KB
✓ Bottle Manifest libnghttp3 (1.15.0)             Downloaded 7.3KB/ 7.3KB
✓ Bottle Manifest ca-certificates (2026-05-14)    Downloaded 1.7KB/ 1.7KB
✓ Bottle Manifest libngtcp2 (1.22.1)              Downloaded 9.7KB/ 9.7KB
✓ Bottle Manifest openssl@3 (3.6.2)              Downloaded 12.0KB/ 12.0KB
✓ Bottle Manifest fmt (12.1.0)                   Downloaded 282.9KB/282.9KB
✓ Bottle Manifest ada-url (3.4.4)                 Downloaded 345.6KB/345.6KB
✓ Bottle Manifest libnghttp3 (1.15.0)             Downloaded 188.8KB/188.8KB
✓ Bottle Manifest libuv (1.52.1)                 Downloaded 7.5KB/ 7.5KB
```

Рис. 1: Установка утилиты

После установки была выполнена первоначальная настройка Atlas CLI:

```
atlas setup
```

В процессе настройки была выполнена авторизация в аккаунте MongoDB Atlas и установка оболочки `mongosh` для работы с базой данных MongoDB.

Для проверки успешной авторизации использовалась команда:

```
atlas auth whoami
```

Команда позволяет определить текущий аккаунт MongoDB Atlas, под которым выполнен вход в систему.

```
6ogdanmezentsev@MacBook-Air ~ % atlas auth whoami
Logged in as mezentsev.6ogdan@gmail.com account
```

Рис. 2: Проверка авторизации

Подключение к кластеру (который был создан в ходе авторизации: `Cluster46053`) выполнялось с помощью команды:

```
atlas clusters connect Cluster46053
```

После выполнения команды была открыта оболочка MongoDB Shell (mongosh) и установлено подключение к кластеру MongoDB Atlas.

```
[6ogdanmezentsev@MacBook-Air ~ % atlas clusters connect Cluster46053
? How would you like to connect to Cluster46053? [Use arrows to move, type to filter]
> mongosh - MongoDB Shell
compass - MongoDB Compass
vscode - MongoDB for VsCode
connectionString - Connection String
```

Рис. 3: Подключение к кластеру

```
[6ogdanmezentsev@MacBook-Air ~ % atlas clusters connect Cluster46053
? How would you like to connect to Cluster46053? mongosh
[? Username for authenticating to MongoDB cluster Cluster46053
[? Password for authenticating to MongoDB cluster ****
Current Mongosh Log ID: 6a08be396a78f7741c36e11c
Connecting to:      mongodb+srv://<credentials>@cluster46053.yr17pzo.mongodb.net/?appName=mongosh+2.8.3
Using MongoDB:      8.0.23
Using Mongosh:       2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/
```

Рис. 4: Успешное подключение к кластеру

Видим, что подключение выполнено успешно, но сейчас используется БД с названием **test** — это стандартная база данных MongoDB по умолчанию.

4.2 Вызов базовых методов и создание БД

Для просмотра списка доступных методов объекта базы данных использовалась команда:

```
db.help()
```

Команда отображает основные функции и методы для работы с MongoDB.

```

Atlas atlas-12dqp-d-shard-0 [primary] test> db.help()

Database Class:

getMongo                Returns the current database connection
getName                 Returns the name of the DB
getCollectionNames      Returns an array containing the names of all collections in the current database.
getCollectionInfos      Returns an array of documents with collection information, i.e. collection name and options, for
se.
runCommand              Runs an arbitrary command on the database.
adminCommand            Runs an arbitrary command against the admin database.
aggregate               Runs a specified admin/diagnostic pipeline which does not require an underlying collection.
getSiblingDB            Returns another database without modifying the db variable in the shell environment.
getCollection            Returns a collection or a view object that is functionally equivalent to using the db.<collection>
dropDatabase            Removes the current database, deleting the associated data files.
createUser              Creates a new user for the database on which the method is run. db.createUser() returns a duplic
he user already exists on the database.
updateUser              Updates the user's profile on the database on which you run the method. An update to a field com
e previous field's values. This includes updates to the user's roles array.
changeUserPassword      Updates a user's password. Run the method in the database where the user is defined, i.e. the da
the user.
logout                  Ends the current authentication session. This function has no effect if the current session is r
dropUser                Removes the user from the current database.
dropAllUsers            Removes all users from the current database.
auth                    Allows a user to authenticate to the database from within the shell.
grantRolesToUser         Grants additional roles to a user.
revokeRolesFromUser     Removes a one or more roles from a user on the current database.
getUser                 Returns user information for a specified user. Run this method on the user's database. The user
atabase on which the method runs.
getUsers                Returns information for all the users in the database.
createCollection         Create new collection
createEncryptedCollection Creates a new collection with a list of encrypted fields each with unique and auto-created data
Ks). This is a utility function that internally utilises ClientEncryption.createEncryptedCollection.
createView              Create new view
createRole               Creates a new role.
updateRole               Updates the role's profile on the database on which you run the method. An update to a field com
e previous field's values.
dropRole                Removes the role from the current database.
dropAllRoles            Removes all roles from the current database.
grantRolesToRole         Grants additional roles to a role.
revokeRolesFromRole     Removes a one or more roles from a role on the current database.
grantPrivilegesToRole   Grants additional privileges to a role.
revokePrivilegesFromRole Removes a one or more privileges from a role on the current database

```

Рис. 5: Вызов метода db.help()

```

Atlas atlas-12dqp-d-shard-0 [primary] test> db.stats()
{
  db: 'test',
  collections: 0,
  views: 0,
  objects: Long('0'),
  avgObjSize: 0,
  dataSize: Long('0'),
  storageSize: Long('0'),
  totalFreeStorageSize: Long('0'),
  numExtents: Long('0'),
  indexes: 0,
  indexSize: Long('0'),
  indexFreeStorageSize: Long('0'),
  fileSize: Long('0'),
  nsSizeMB: 0,
  ok: 1
}

```

Рис. 6: Вызов метода db.stats()

Метод `stats()` используется для получения статистики о базе данных или коллекции. Как видим, из-за того, что бд `test` пустая, то статистики нет.

Для создания базы данных использовалась команда:

```
use learn
```

Команда `use` переключает текущий контекст на указанную базу данных.

```
[Atlas atlas-12dqp-d-shard-0 [primary] test> use learn
switched to db learn
Atlas atlas-12dqp-d-shard-0 [primary] learn> █
```

Рис. 7: Переключение контекста на новую БД

Для просмотра списка всех существующих БД была использована команда:

```
show dbs
```

В результате выполнения команды выводятся имена баз данных и объем занимаемого ими дискового пространства.

```
[Atlas atlas-12dqp-d-shard-0 [primary] learn> show dbs
sample_airbnb      83.75 MiB
sample_analytics  14.49 MiB
sample_geospatial 1.84 MiB
sample_guides     56.00 KiB
sample_mflix      116.28 MiB
sample_restaurants 11.09 MiB
sample_supplies    1.95 MiB
sample_training    64.32 MiB
sample_weatherdata 4.18 MiB
admin              0 B
local              0 B
Atlas atlas-12dqp-d-shard-0 [primary] learn> █
```

Рис. 8: Результат выполнения команды show dbs

В списке отображаются учебные и системные базы данных MongoDB Atlas. Базы данных с префиксом **sample_** являются демонстрационными наборами данных, автоматически добавляемыми MongoDB Atlas для обучения и тестирования запросов.

Системные базы данных:

admin — Системная база данных MongoDB, предназначенная для хранения информации о пользователях, ролях и административных настройках сервера.

local — Системная база данных, используемая MongoDB для хранения служебной информации replica set и внутренних данных сервера.

База данных **learn** ещё не отображалась в списке, поскольку MongoDB создает базы данных физически только после добавления первого документа или коллекции.

4.3 Работа с коллекциями и документами

Для создания коллекции unicorns была использована такая команда:

```
db.unicorns.insertOne({name: 'Aurora', gender: 'f',  
weight: 450})
```

```
[Atlas atlas-12dqp-d-shard-0 [primary] learn> db.unicorns.insertOne({name: 'Aurora', gender: 'f', weight: 450})  
{  
  acknowledged: true,  
  insertedId: ObjectId('6a09948c6a78f7741c36e11d')  
}
```

Рис. 9: Создание коллекции unicorns

Для просмотра коллекций в БД была использована команда:

```
show collections
```

```
[Atlas atlas-12dqp-d-shard-0 [primary] learn> show collections  
unicorns  
Atlas atlas-12dqp-d-shard-0 [primary] learn> █
```

Рис. 10: Получение списка коллекций

Для переименования коллекции была использована команда:

```
db.unicorns.renameCollection("horses")
```

```
[Atlas atlas-12dqp-d-shard-0 [primary] learn> db.unicorns.renameCollection("horses")  
{ ok: 1 }  
Atlas atlas-12dqp-d-shard-0 [primary] learn> █
```

Рис. 11: Успешное переименование коллекции

Для получения статистики коллекции была использована команда:

```
db.horses.stats()
```

```

[Atlas atlas-12dqp-d-shard-0 [primary] learn> db.horses.stats()
{
  ok: 1,
  capped: false,
  wiredTiger: {
    metadata: { formatVersion: 1 },
    creationString: 'access_pattern_hint=none,allocation_size=4KB,app_metadata=(formatVersion=1),assert=(comp=none,write_timestamp=off),block_allocation=best,block_compressor=snappy,cache_resident=false,checksum=on,(keyid=,name=),exclusive=false,extractor=,format=btree,huffman_key=,huffman_value=,ignore_in_memory_cache_size=,internal_item_max=,internal_page_max=4KB,key_format=q,key_gap=10,leaf_item_max=0,leaf_key_max=0,leaf_page_max=32KB,leaf_value_max=64KB,bloom_bit_count=16,bloom_config=,bloom_hash_count=8,bloom_oldest=false,chunk_count_limit=0,chunk_max=56B,n=0,suffix=),merge_max=15,merge_min=0),memory_page_image_max=0,memory_page_max=10m,os_cache_dirty_max=0,os_cache_min=4,source=,split_deepen_min_child=0,split_deepen_per_child=0,split_pct=90,tiered_storage=(auth_token=,buffer=,object_target_size=0),type=file,value_format=u,verbose=[],write_timestamp_usage=none',
    type: 'file',
    uri: 'statistics:table:collection-12617-1646100060075981544',
    LSM: {
      'bloom filter false positives': 0,
      'bloom filter hits': 0,
      'bloom filter misses': 0,
      'bloom filter pages evicted from cache': 0,
      'bloom filter pages read into cache': 0,
      'bloom filters in the LSM tree': 0,
      'chunks in the LSM tree': 0,
      'highest merge generation in the LSM tree': 0,
      'queries that could have benefited from a Bloom filter that did not exist': 0,
      'sleep for LSM checkpoint throttle': 0,
      'sleep for LSM merge throttle': 0,
      'total size of bloom filters': 0
    },
    autocommit: {
      'retries for readonly operations': 0,
      'retries for update operations': 0
    },
    backup: {
      'total modified incremental blocks with compressed data': 0,
    },
    session: { 'object compaction': 0 },
    transaction: {
      'a reader raced with a prepared transaction commit and skipped an update or updates': 0,
      'number of times overflow removed value is read': 0,
      'race to read prepared update retry': 0,
      'rollback to stable history store keys that would have been swept in non-dryrun mode': 0,
      'rollback to stable history store records with stop timestamps older than newer records': 0,
      'rollback to stable inconsistent checkpoint': 0,
      'rollback to stable keys removed': 0,
      'rollback to stable keys restored': 0,
      'rollback to stable keys that would have been removed in non-dryrun mode': 0,
      'rollback to stable keys that would have been restored in non-dryrun mode': 0,
      'rollback to stable restored tombstones from history store': 0,
      'rollback to stable restored updates from history store': 0,
      'rollback to stable skipping delete rle': 0,
      'rollback to stable skipping stable rle': 0,
      'rollback to stable sweeping history store keys': 0,
      'rollback to stable tombstones from history store that would have been restored in non-dryrun mode': 0,
      'rollback to stable updates from history store that would have been restored in non-dryrun mode': 0,
      'rollback to stable updates removed from history store': 0,
      'rollback to stable updates that would have been removed from history store in non-dryrun mode': 0,
      'update conflicts': 0
    }
  },
  sharded: false,
  size: 65,
  count: 1,
  numOrphanDocs: 0,
  storageSize: 36864,
  totalIndexSize: 36864,
  totalSize: 73728,
  indexSizes: { _id_: 36864 },
  avgObjSize: 65,
  ns: 'learn.horses',
  nindexes: 1,
  scaleFactor: 1
}
Atlas atlas-12dqp-d-shard-0 [primary] learn>

```

Рис. 12-13: Успешное получение статистики коллекции

Для удаления коллекции была использована команда:

```
db.horses.drop()
```

```
[Atlas atlas-12dqpd-shard-0 [primary] learn> db.horses.drop()  
true  
Atlas atlas-12dqpd-shard-0 [primary] learn> █
```

Рис. 14: Успешное удаление коллекции

Для удаления БД была использована команда:

```
db.dropDatabase()
```

```
[Atlas atlas-12dqpd-shard-0 [primary] learn> db.dropDatabase()  
{ ok: 1, dropped: 'learn' }  
Atlas atlas-12dqpd-shard-0 [primary] learn> █
```

Рис. 15: Успешное удаление БД

5 Выполнение лабораторной работы №6.2

5.1 Вставка документов в коллекцию

Выполнение практического задания 2.1.1

С помощью команды use была создана БД learn. После этого, используя первый способ для добавления документов в коллекцию из описания ЛР 6.2, в коллекцию unicorns были добавлены документы:

```
db.unicorns.insert({name: 'Horny', loves: ['carrot', 'papaya'],  
weight: 600, gender: 'm', vampires: 63});  
db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'],  
weight: 450, gender: 'f', vampires: 43});  
db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'],  
weight: 984, gender: 'm', vampires: 182});  
db.unicorns.insert({name: 'Roooooodles', loves: ['apple'], weight:  
575, gender: 'm', vampires: 99});  
db.unicorns.insert({name: 'Solnara', loves: ['apple', 'carrot',  
'chocolate'], weight: 550, gender: 'f', vampires: 80});  
db.unicorns.insert({name: 'Ayna', loves: ['strawberry', 'lemon'],  
weight: 733, gender: 'f', vampires: 40});  
db.unicorns.insert({name: 'Kenny', loves: ['grape', 'lemon'], weight:  
690, gender: 'm', vampires: 39});  
db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'],  
weight: 421, gender: 'm', vampires: 2});  
db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'],  
weight: 601, gender: 'f', vampires: 33});
```

```

    db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'],
weight: 650, gender: 'm', vampires: 54});
    db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'],
weight: 540, gender: 'f'});

```

```

[Atlas atlas-12dqp-shard-0 [primary] learn> db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a12e594e66b8b0ed7469a52') }
}

```

Рис. 16: Добавление документа в коллекцию первым способом

И также используя второй способ из описания, был добавлен еще один документ:

```

[Atlas atlas-12dqp-shard-0 [primary] learn> document=({name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165})
{
  name: 'Dunx',
  loves: [ 'grape', 'watermelon' ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
[Atlas atlas-12dqp-shard-0 [primary] learn> db.unicorns.insert(document)
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a12e79de66b8b0ed7469a53') }
}

```

Рис. 17: Добавление документа в коллекцию вторым способом

После этого была сделана проверка содержимого коллекции с помощью метода find():

```

[Atlas atlas-12dqp-shard-0 [primary] learn> db.unicorns.find({}, {name: 1, _id: 1})
[
  { _id: ObjectId('6a12e249e66b8b0ed7469a47'), name: 'Horny' },
  { _id: ObjectId('6a12e329e66b8b0ed7469a48'), name: 'Aurora' },
  { _id: ObjectId('6a12e33ae66b8b0ed7469a49'), name: 'Unicrom' },
  { _id: ObjectId('6a12e343e66b8b0ed7469a4a'), name: 'Rooooooodles' },
  { _id: ObjectId('6a12e34be66b8b0ed7469a4b'), name: 'Solnara' },
  { _id: ObjectId('6a12e351e66b8b0ed7469a4c'), name: 'Ayna' },
  { _id: ObjectId('6a12e356e66b8b0ed7469a4d'), name: 'Kenny' },
  { _id: ObjectId('6a12e35ce66b8b0ed7469a4e'), name: 'Raleigh' },
  { _id: ObjectId('6a12e362e66b8b0ed7469a4f'), name: 'Leia' },
  { _id: ObjectId('6a12e367e66b8b0ed7469a50'), name: 'Pilot' },
  { _id: ObjectId('6a12e594e66b8b0ed7469a52'), name: 'Nimue' },
  { _id: ObjectId('6a12e79de66b8b0ed7469a53'), name: 'Dunx' }
]

```

Рис. 18: Проверка содержимого коллекции с помощью метода find

5.2 Выборка данных из БД

Выполнение практического задания 2.2.1

В ходе выполнения задания были сформированы запросы выборки документов из коллекции `unicorns` с использованием метода `find()`.

1) Получение списка самцов:

```
[Atlas atlas-12dqp-d-shard-0 [primary] learn> db.unicorns.find({gender: 'm'}).sort({name: 1})
[
  {
    _id: ObjectId('6a12e79de66b8b0ed7469a53'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a12e249e66b8b0ed7469a47'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a12e356e66b8b0ed7469a4d'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a12e367e66b8b0ed7469a50'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a12e35ce66b8b0ed7469a4e'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a12e343e66b8b0ed7469a4a'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a12e33ae66b8b0ed7469a49'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  }
]
```

Рис. 19: Список всех самцов с сортировкой по имени

Запрос выполняет выборку всех самцов единорогов и сортирует результат по имени в алфавитном порядке.

2) Получение первых трех самок:

```
[Atlas atlas-12dqp-d-shard-0 [primary] learn> db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
[
  {
    _id: ObjectId('6a12e329e66b8b0ed7469a48'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a12e351e66b8b0ed7469a4c'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a12e362e66b8b0ed7469a4f'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]
```

Рис. 20: Список первых трех самок с сортировкой по имени

Запрос выводит только первые три документа после сортировки списка самок по имени.

3) Получение всех самок, любящих carrot:

```
[Atlas atlas-12dqp-d-shard-0 [primary] learn> db.unicorns.find({gender: 'f', loves: 'carrot'})
[
  {
    _id: ObjectId('6a12e329e66b8b0ed7469a48'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a12e34be66b8b0ed7469a4b'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a12e594e66b8b0ed7469a52'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

Рис. 21: Список всех самок, любящих carrot

4) Получение только первой такой самки через findOne:

```
[Atlas atlas-12dqp-d-shard-0 [primary] learn> db.unicorns.findOne({gender: 'f', loves: 'carrot'})
{
  _id: ObjectId('6a12e329e66b8b0ed7469a48'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
```

Рис. 22: Первая самка, любящая carrot, через findOne

5) Получение только первой такой самки через limit:

```
[Atlas atlas-12dqp-d-shard-0 [primary] learn> db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
[
  {
    _id: ObjectId('6a12e329e66b8b0ed7469a48'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
```

Рис. 23: Первая самка, любящая carrot, через limit

Выполнение практического задания 2.2.2

Модифицируйте запрос для вывода списков самцов единорогов, исключив из результата информацию о предпочтениях.

```
[Atlas atlas-12dqp-d-shard-0 [primary] learn> db.unicorns.find({gender: 'm'}, {loves: 0}).sort({name: 1})
[
  {
    _id: ObjectId('6a12e79de66b8b0ed7469a53'),
    name: 'Dunx',
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a12e249e66b8b0ed7469a47'),
    name: 'Horny',
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a12e356e66b8b0ed7469a4d'),
    name: 'Kenny',
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a12e367e66b8b0ed7469a50'),
    name: 'Pilot',
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a12e35ce66b8b0ed7469a4e'),
    name: 'Raleigh',
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a12e343e66b8b0ed7469a4a'),
    name: 'Rooooooodles',
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a12e33ae66b8b0ed7469a49'),
    name: 'Unicrom',
    weight: 984,
    gender: 'm',
    vampires: 182
  }
]
```

Рис. 24: Список самцов, исключая информацию о предпочтениях

Выполнение практического задания 2.2.3

Вывести список единорогов в обратном порядке добавления.

```
[Atlas atlas-12dqp-d-shard-0 [primary] learn> db.unicorns.find().sort({$natural: -1})
[
  {
    _id: ObjectId('6a12e79de66b8b0ed7469a53'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a12e594e66b8b0ed7469a52'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a12e367e66b8b0ed7469a50'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a12e362e66b8b0ed7469a4f'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('6a12e35ce66b8b0ed7469a4e'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  }
]
```

Рис. 25: Список единорогов в обратном порядке добавления

Выполнение практического задания 2.2.4

Вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор.

```
[Atlas atlas-12dqp-d-shard-0 [primary] learn> db.unicorns.find({}, {_id: 0, name: 1, loves: {$slice: 1}})
[
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Aurora', loves: [ 'carrot' ] },
  { name: 'Unicrom', loves: [ 'energon' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Solnara', loves: [ 'apple' ] },
  { name: 'Ayna', loves: [ 'strawberry' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Leia', loves: [ 'apple' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Nimue', loves: [ 'grape' ] },
  { name: 'Dunx', loves: [ 'grape' ] }
]
```

Рис. 26: Список единорогов с названием первого предпочтения, исключая поле `_id`

В ходе выполнения задания была выполнена выборка документов из коллекции `unicorns` с использованием проекции полей.

Для массива `loves` был применен оператор `$slice`, позволяющий вывести только первый элемент массива предпочтений. Также из результата был исключен идентификатор `_id`.

5.3 Логические операторы

Выполнение практического задания 2.3.1

Вывести список самок единорогов весом от полутонны до 700 кг, исключив вывод идентификатора.

```
[Atlas atlas-12dqp-d-shard-0 [primary] learn> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0})
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

Рис. 27: Список единорогов весом от 500 кг до 700 кг, исключая поле `_id`

В ходе выполнения задания были использованы логические операторы сравнения `$gte` и `$lte`.

Выполнение практического задания 2.3.2

Вывести список самцов единорогов весом от полутонны и предпочитающих `grape` и `lemon`, исключив вывод идентификатора.

```
[Atlas atlas-12dqpd-shard-0 [primary] learn> db.unicorns.find({gender: 'm', weight: {$gte: 500}, loves: {$all: ['grape', 'lemon']}}, {_id: 0})
[
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
```

Рис. 28: Список самцов весом от 500 кг и предпочитающих `grape` и `lemon`, исключая поле `_id`

В ходе выполнения задания был использован оператор `$all`, предназначенный для поиска документов, массив которых содержит все указанные элементы.

Выполнение практического задания 2.3.3

Найти всех единорогов, не имеющих ключ `vampires`.

```
[Atlas atlas-12dqpd-shard-0 [primary] learn> db.unicorns.find({vampires: {$exists: false}})
[
  {
    _id: ObjectId('6a12e594e66b8b0ed7469a52'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

Рис. 29: Список единорогов, не имеющих поле `vampires`

В ходе выполнения задания был использован оператор `$exists`, позволяющий проверить наличие или отсутствие поля в документе.

Выполнение практического задания 2.3.4

Вывести список упорядоченный список имен самцов единорогов с информацией об их первом предпочтении.

```
[Atlas atlas-12dqp-d-shard-0 [primary] learn> db.unicorns.find({gender: 'm'}, {_id: 0, name: 1, loves: {$slice: 1}}).sort({name: 1})
[
  { name: 'Dunx', loves: [ 'grape' ] },
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Roooooodles', loves: [ 'apple' ] },
  { name: 'Unicrom', loves: [ 'energon' ] }
]
```

Рис. 30: Список самцов с информацией об их первом предпочтении, сортировка по имени

5.4 Запросы к вложенным объектам

Выполнение практического задания 3.1.1

1) В ходе выполнения задания была создана коллекция **towns**, содержащая вложенные документы **mayor**. Были добавлены сведения о городах, населении, известных особенностях и мэрах.

```
Atlas atlas-12dqp-d-shard-0 [primary] learn> db.towns.insertMany([
  {
    name: "Punxsutawney",
    population: 6200,
    last_sensus: ISODate("2008-01-31"),
    famous_for: ["phil the groundhog"],
    mayor: {
      name: "Jim Wehrle"
    }
  },
  {
    name: "New York",
    population: 22200000,
    last_sensus: ISODate("2009-07-31"),
    famous_for: ["statue of liberty", "food"],
    mayor: {
      name: "Michael Bloomberg",
      party: "I"
    }
  },
  {
    name: "Portland",
    population: 528000,
    last_sensus: ISODate("2009-07-20"),
    famous_for: ["beer", "food"],
    mayor: {
      name: "Sam Adams",
      party: "D"
    }
  }
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a131381e66b8b0ed7469a54'),
    '1': ObjectId('6a131381e66b8b0ed7469a55'),
    '2': ObjectId('6a131381e66b8b0ed7469a56')
  }
}
```

Рис. 31: Создание коллекции towns и добавление документов

2) Сформировать запрос, который возвращает список городов с независимыми мэрами (party="I") . Вывести только название города и информацию о мэре.

```
[Atlas atlas-12dqp-d-shard-0 [primary] learn> db.towns.find({"mayor.party": "I"}, {_id: 0, name: 1, mayor: 1})  
[  
  { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } }  
]
```

Рис. 32: Список городов с независимыми мэрами

В ходе выполнения задания был сформирован запрос к вложенному объекту с использованием оператора точки. Были выбраны города, мэры которых являются независимыми (party = "I").

3) Сформировать запрос, который возвращает список беспартийных мэров (party отсутствует) . Вывести только название города и информацию о мэре.

```
[Atlas atlas-12dqp-d-shard-0 [primary] learn> db.towns.find({"mayor.party": {$exists: false}}, {_id: 0, name: 1, mayor: 1})  
[ { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]  
Atlas atlas-12dqp-d-shard-0 [primary] learn> █
```

Рис. 33: Список городов с беспартийными мэрами

В ходе выполнения задания был использован оператор `$exists` для поиска документов, в которых отсутствует поле `party` у вложенного объекта `mayor`.

Выполнение практического задания 3.1.2

- 1) Сформировать функцию для вывода списка самцов единорогов.
- 2) Создать курсор для этого списка из первых двух особей с сортировкой в лексикографическом порядке.
- 3) Вывести результат, используя `forEach`

Функция для вывода самцов:

```
Atlas atlas-12dqp-d-shard-0 [primary] learn> function showMaleUnicorns() {
|   return db.unicorns.find({gender: 'm'})
| }
[Function: showMaleUnicorns]
[Atlas atlas-12dqp-d-shard-0 [primary] learn> showMaleUnicorns()
[
  {
    _id: ObjectId('6a12e249e66b8b0ed7469a47'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a12e33ae66b8b0ed7469a49'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a12e343e66b8b0ed7469a4a'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a12e356e66b8b0ed7469a4d'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
  }
```

Рис. 34: Создание и вызов функции для вывода списка единорогов самцов

В ходе выполнения задания была создана JavaScript-функция для получения списка самцов единорогов.

Создание курсора:

```
[Atlas atlas-12dqp-d-shard-0 [primary] learn> var cursor = showMaleUnicorns().sort({name: 1}).limit(2)
[Atlas atlas-12dqp-d-shard-0 [primary] learn> ]
```

Рис. 35: Создание курсора на основе функции

Вывод через forEach:

```
Atlas atlas-12dqp-d-shard-0 [primary] learn> cursor.forEach(function(obj){
|   print(obj.name)
| })
Dunx
Horny
```

Рис. 36: Вывод результата с помощью метода forEach

На основе функции был создан курсор с сортировкой по имени и ограничением количества документов. Для вывода результатов использовался метод `forEach()`.

5.5 Агрегированные запросы

Выполнение практического задания 3.2.1

Вывести количество самок единорогов весом от полутонны до 600 кг.

```
Atlas atlas-12dqp-d-shard-0 [primary] learn> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 600}}).count()
2
Atlas atlas-12dqp-d-shard-0 [primary] learn> █
```

Рис. 37: Вывод количества самок единорогов весом от 500 кг и до 600 кг

Выполнение практического задания 3.2.2

Вывести список уникальных предпочтений единорогов.

```
Atlas atlas-12dqp-d-shard-0 [primary] learn> db.unicorns.distinct("loves")
[
  'apple',      'carrot',
  'chocolate', 'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
```

Рис. 38: Вывод списка уникальных предпочтений единорогов

В ходе выполнения задания был использован метод `distinct()` для получения уникальных значений предпочтений единорогов.

Выполнение практического задания 3.2.3

Посчитать количество особей единорогов обоих полов.

```
[Atlas atlas-12dqp-shard-0 [primary] learn> db.unicorns.aggregate([{$group: {_id: "$gender", count: {$sum: 1}}}}])
[ { _id: 'f', count: 5 }, { _id: 'm', count: 7 } ]
Atlas atlas-12dqp-shard-0 [primary] learn> █
```

Рис. 39: Вывод количества самцов и самок

В ходе выполнения задания был использован агрегатный запрос `aggregate()` с оператором `$group` для подсчета количества документов по полю `gender`.

5.6 Редактирование данных

Выполнение практического задания 3.3.1

1. *Выполнить команду:*

```
> db.unicorns.save({name: 'Barney', loves: ['grape'],
weight: 340, gender: 'm'})
```

2. *Проверить содержимое коллекции `unicorns`.*

В более новой версии MongoDB не был найден метод `save()`

```
Atlas atlas-12dqp-shard-0 [primary] learn> db.unicorns.save({name: 'Barney', loves: ['grape'],
[| weight: 340, gender: 'm']})
TypeError: db.unicorns.save is not a function
Atlas atlas-12dqp-shard-0 [primary] learn> █
```

Выполнение практического задания 3.3.2

1. *Для самки единорога Айна внести изменения в БД: теперь ее вес 800, она убила 51 вампира.*

2. *Проверить содержимое коллекции `unicorns`.*

```

Atlas atlas-12dqp-shard-0 [primary] learn> db.unicorns.updateOne(
  {
    name: 'Ayna',
    $set: {
      weight: 800,
      vampires: 51
    }
  }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
[Atlas atlas-12dqp-shard-0 [primary] learn> db.unicorns.find({name: 'Ayna'})
[
  {
    _id: ObjectId('6a12e351e66b8b0ed7469a4c'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 800,
    gender: 'f',
    vampires: 51
  }
]

```

Рис. 40: Изменение документа и проверка содержимого

В ходе выполнения задания был использован оператор `$set` для изменения отдельных полей документа.

Выполнение практического задания 3.3.3

1. Для самца единорога Raleigh внести изменения в БД: теперь он любит рэдбул.
2. Проверить содержимое коллекции `unicorns`.

```

Atlas atlas-12dqp-shard-0 [primary] learn> db.unicorns.updateOne(
  {
    name: 'Raleigh',
    $push: {
      loves: 'redbull'
    }
  }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
[Atlas atlas-12dqp-shard-0 [primary] learn> db.unicorns.find({name: 'Raleigh'})
[
  {
    _id: ObjectId('6a12e35ce66b8b0ed7469a4e'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar', 'redbull' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  }
]

```

Рис. 41: Изменение документа и проверка содержимого

В ходе выполнения задания был использован оператор **\$push** для добавления нового элемента в массив **loves**.

Выполнение практического задания 3.3.4

1. *Всем самцам единорогов увеличить количество убитых вампиров на 5.*
2. *Проверить содержимое коллекции `unicorns`.*

```
Atlas atlas-12dqp-d-shard-0 [primary] learn> db.unicorns.updateMany(
|   {gender: 'm'},
|   {
|     $inc: {
|       vampires: 5
|     }
|   }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 7,
  modifiedCount: 7,
  upsertedCount: 0
}
Atlas atlas-12dqp-d-shard-0 [primary] learn> db.unicorns.find(
|   {gender: 'm'},
|   {
|     _id: 0,
|     name: 1,
|     vampires: 1
|   }
| )
[
  { name: 'Horny', vampires: 68 },
  { name: 'Unicrom', vampires: 187 },
  { name: 'Rooooooodles', vampires: 104 },
  { name: 'Kenny', vampires: 44 },
  { name: 'Raleigh', vampires: 7 },
  { name: 'Pilot', vampires: 59 },
  { name: 'Dunx', vampires: 170 }
]
```

Рис. 42: Изменение документа и проверка содержимого

В ходе выполнения задания был использован оператор **\$inc** для увеличения числового значения поля **vampires**.

Выполнение практического задания 3.3.5

1. *Изменить информацию о городе Портланд: мэр этого города теперь беспартийный.*
2. *Проверить содержимое коллекции `towns`.*

```

Atlas atlas-12dqp-shard-0 [primary] learn> db.towns.updateOne(
|   {name: 'Portland'},
|   {
|     $unset: {
|       "mayor.party": 1
|     }
|   }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
[Atlas atlas-12dqp-shard-0 [primary] learn> db.towns.find({name: 'Portland'})
[
  {
    _id: ObjectId('6a131381e66b8b0ed7469a56'),
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]

```

Рис. 43: Изменение документа и проверка содержимого

В ходе выполнения задания был использован оператор `$unset` для удаления поля `party` из вложенного объекта `mayor`.

Выполнение практического задания 3.3.6

1. *Изменить информацию о самце единорога Pilot: теперь он любит и шоколад.*
2. *Проверить содержимое коллекции unicorns.*

```

Atlas atlas-12dqp-shard-0 [primary] learn> db.unicorns.updateOne(
|   {name: 'Pilot'},
|   {
|     $push: {
|       loves: 'chocolate'
|     }
|   }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
[Atlas atlas-12dqp-shard-0 [primary] learn> db.unicorns.find({name: 'Pilot'})
[
  {
    _id: ObjectId('6a12e367e66b8b0ed7469a50'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  }
]

```

Рис. 44: Изменение документа и проверка содержимого

В ходе выполнения задания был использован оператор `$push` для добавления нового элемента в массив предпочтений.

Выполнение практического задания 3.3.7

1. Изменить информацию о самке единорога Aurora: теперь она любит еще и сахар, и лимоны.
2. Проверить содержимое коллекции unicorns.

```
Atlas atlas-12dqp-d-shard-0 [primary] learn> db.unicorns.updateOne(
|   {name: 'Aurora'},
|   {
|     $addToSet: {
|       loves: {
|         $each: ['sugar', 'lemon']
|       }
|     }
|   }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
[Atlas atlas-12dqp-d-shard-0 [primary] learn> db.unicorns.find({name: 'Aurora'})
[
  {
    _id: ObjectId('6a12e329e66b8b0ed7469a48'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
```

Рис. 45: Изменение документа и проверка содержимого

В ходе выполнения задания был использован оператор `$addToSet` совместно с `$each` для добавления нескольких уникальных элементов в массив.

5.7 Удаление данных

Выполнение практического задания 3.4.1

- 1) Удаление городов с беспартийными мэрами

```
Atlas atlas-12dqp-d-shard-0 [primary] learn> db.towns.remove({
|   "mayor.party": {$exists: false}
| })
DeprecationWarning: Collection.remove() is deprecated. Use deleteOne, deleteMany, findOneAndDelete, or bulkWrite.
{ acknowledged: true, deletedCount: 2 }
[Atlas atlas-12dqp-d-shard-0 [primary] learn> db.towns.find()
[
  {
    _id: ObjectId('6a131381e66b8b0ed7469a55'),
    name: 'New York',
    population: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'statue of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  }
]
```

Рис. 46: Удаление данных и проверка содержимого

2) Очистка коллекции towns

```
[Atlas atlas-12dqpд-shard-0 [primary] learn> db.towns.remove({})
{ acknowledged: true, deletedCount: 1 }
[Atlas atlas-12dqpд-shard-0 [primary] learn> db.towns.find()

Atlas atlas-12dqpд-shard-0 [primary] learn> █
```

Рис. 47: Очистка коллекции и проверка содержимого

3) Просмотр коллекций

```
[Atlas atlas-12dqpд-shard-0 [primary] learn> show collections
towns
unicorns
```

Рис. 48: Получение списка коллекций

В ходе выполнения задания была выполнена полная очистка коллекции **towns** и просмотр списка существующих коллекций базы данных.

5.8 Ссылки в MongoDB

Выполнение практического задания 4.1.1

- 1) *Создайте коллекцию зон обитания единорогов, указав в качестве идентификатора кратко название зоны, далее включив полное название и описание.*
- 2) *Включите для нескольких единорогов в документы ссылку на зону обитания, используя второй способ автоматического связывания.*
- 3) *Проверьте содержание коллекции единорогов.*

1) Создание коллекции habitats

```
Atlas atlas-12dqpд-shard-0 [primary] learn> db.habitats.insertMany([
| {
|   _id: "forest",
|   full_name: "Mystic Forest",
|   description: "Magic forest with ancient trees"
| },
| {
|   _id: "mountain",
|   full_name: "Crystal Mountains",
|   description: "Cold mountains with crystal caves"
| }
| ])
{ acknowledged: true, insertedIds: { '0': 'forest', '1': 'mountain' } }
```

Рис. 49: Создание коллекции habitats

В ходе выполнения задания была создана коллекция **habitats**, содержащая информацию о зонах обитания единорогов.

2) Добавление ссылок

```
Atlas atlas-12dqp-d-shard-0 [primary] learn> db.unicorns.updateOne(
  {name: 'Horny'},
  {
    $set: {
      habitat: {
        $ref: "habitats",
        $id: "forest"
      }
    }
  }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
Atlas atlas-12dqp-d-shard-0 [primary] learn> db.unicorns.updateOne(
  {name: 'Pilot'},
  {
    $set: {
      habitat: {
        $ref: "habitats",
        $id: "mountain"
      }
    }
  }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Рис. 50: Добавление ссылок для двух единорогов

3) Проверка содержимого коллекции

```
Atlas atlas-12dqp-d-shard-0 [primary] learn> db.unicorns.find(
  {},
  {
    name: 1,
    habitat: 1
  }
)
[
  {
    _id: ObjectId('6a12e249e66b8b0ed7469a47'),
    name: 'Horny',
    habitat: DBRef('habitats', 'forest')
  },
  { _id: ObjectId('6a12e329e66b8b0ed7469a48'), name: 'Aurora' },
  { _id: ObjectId('6a12e33ae66b8b0ed7469a49'), name: 'Unicrom' },
  { _id: ObjectId('6a12e343e66b8b0ed7469a4a'), name: 'Rooooooodles' },
  { _id: ObjectId('6a12e34be66b8b0ed7469a4b'), name: 'Solnara' },
  { _id: ObjectId('6a12e351e66b8b0ed7469a4c'), name: 'Ayna' },
  { _id: ObjectId('6a12e356e66b8b0ed7469a4d'), name: 'Kenny' },
  { _id: ObjectId('6a12e35ce66b8b0ed7469a4e'), name: 'Raleigh' },
  { _id: ObjectId('6a12e362e66b8b0ed7469a4f'), name: 'Leia' },
  {
    _id: ObjectId('6a12e367e66b8b0ed7469a50'),
    name: 'Pilot',
    habitat: DBRef('habitats', 'mountain')
  },
  { _id: ObjectId('6a12e594e66b8b0ed7469a52'), name: 'Nimue' },
  { _id: ObjectId('6a12e79de66b8b0ed7469a53'), name: 'Dunx' }
]
```

Рис. 51: Проверка содержимого коллекции unicorns

В ходе выполнения задания были созданы ссылки между коллекциями с использованием механизма DBRef.

5.9 Настройка индексов

Выполнение практического задания 4.2.1

1. Проверьте, можно ли задать для коллекции *unicorns* индекс для ключа *name* с флагом *unique*.

```
Atlas atlas-12dqp-d-shard-0 [primary] learn> db.unicorns.createIndex(  
|   {name: 1},  
|   {unique: true}  
| )  
name_1
```

Рис. 52: Успешное создание индекса

Сейчас все имена уникальны:

- Horny
- Aurora
- Unicrom
- Barny
- Pilot
- и т.д.

Поэтому индекс должен успешно создастся.

5.9 Управление индексами

Выполнение практического задания 4.3.1

- 1) Получите информацию о всех индексах коллекции *unicorns*.
- 2) Удалите все индексы, кроме индекса для идентификатора.
- 3) Попробуйте удалить индекс для идентификатора.

- 1) Получение информации обо всех индексах

```
[Atlas atlas-12dqp-d-shard-0 [primary] learn> db.unicorns.getIndexes()  
[  
  { v: 2, key: { _id: 1 }, name: '_id_' },  
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }  
]  
Atlas atlas-12dqp-d-shard-0 [primary] learn> █
```

Рис. 53: Получение списка индексов

2) Удаление всех индексов, кроме `_id`

```
[Atlas atlas-12dqp-d-shard-0 [primary] learn> db.unicorns.dropIndexes()
{
  nIndexesWas: 2,
  msg: 'non-_id indexes dropped for collection',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1779980421, i: 3 }),
    signature: {
      hash: Binary.createFromBase64('cryfZBJVY1h+ZTzWk7kalLSJKeg=', 0),
      keyId: Long('7594536424632745985')
    }
  },
  operationTime: Timestamp({ t: 1779980421, i: 3 })
}
[Atlas atlas-12dqp-d-shard-0 [primary] learn> db.unicorns.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_' } ] ]
```

Рис. 54: Удаление индексов и получение списка индексов

В ходе выполнения задания были удалены все пользовательские индексы коллекции `unicorns`. После удаления в коллекции остался только системный индекс `_id`.

3) Попытка удалить индекс `_id`

```
[Atlas atlas-12dqp-d-shard-0 [primary] learn> db.unicorns.dropIndex("_id_")
MongoServerError[InvalidOptions]: cannot drop _id index
Atlas atlas-12dqp-d-shard-0 [primary] learn> █
```

Рис. 55: Получение ошибки при попытке удалить системный индекс `_id`

5.9 План запроса

Выполнение практического задания 4.4.1

- 1) Создайте объемную коллекцию `numbers`, задействовав курсор:

```
for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}
```
- 2) Выберите последних четыре документа.
- 3) Проанализируйте план выполнения запроса 2. Сколько потребовалось времени на выполнение запроса? (по значению параметра `executionTimeMillis`)
- 4) Создайте индекс для ключа `value`.
- 5) Получите информацию о всех индексах коллекции `numbers`.
- 6) Выполните запрос 2.
- 7) Проанализируйте план выполнения запроса с установленным индексом. Сколько потребовалось времени на выполнение запроса?
- 8) Сравните время выполнения запросов с индексом и без. Дайте ответ на вопрос: какой запрос более эффективен?

1) Создание объемной коллекции numbers

```
[Atlas atlas-12dqp-d-shard-0 [primary] learn> let arr = [];

Atlas atlas-12dqp-d-shard-0 [primary] learn> for(let i = 0; i < 100000; i++){
|   arr.push({value: i});
| }
100000
[Atlas atlas-12dqp-d-shard-0 [primary] learn> db.numbers.insertMany(arr)
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a185f25bf50771e40223fc2'),
    '1': ObjectId('6a185f25bf50771e40223fc3'),
    '2': ObjectId('6a185f25bf50771e40223fc4'),
    '3': ObjectId('6a185f25bf50771e40223fc5'),
    '4': ObjectId('6a185f25bf50771e40223fc6'),
    '5': ObjectId('6a185f25bf50771e40223fc7'),
    '6': ObjectId('6a185f25bf50771e40223fc8'),
    '7': ObjectId('6a185f25bf50771e40223fc9'),
    '8': ObjectId('6a185f25bf50771e40223fca'),
    '9': ObjectId('6a185f25bf50771e40223fcb'),
    '10': ObjectId('6a185f25bf50771e40223fcc'),
    '11': ObjectId('6a185f25bf50771e40223fcd'),
    '12': ObjectId('6a185f25bf50771e40223fce'),
    '13': ObjectId('6a185f25bf50771e40223fcf'),
```

Рис. 56: Создание коллекции numbers

2) Выборка последних четырех документов

```
Atlas atlas-12dqp-d-shard-0 [primary] learn> db.numbers.find()
|   .sort({value: -1})
|   .limit(4)
[
  { _id: ObjectId('6a185f25bf50771e4023c661'), value: 99999 },
  { _id: ObjectId('6a185f25bf50771e4023c660'), value: 99998 },
  { _id: ObjectId('6a185f25bf50771e4023c65f'), value: 99997 },
  { _id: ObjectId('6a185f25bf50771e4023c65e'), value: 99996 }
]
```

Рис. 57: Получение последних 4 документов

В ходе выполнения задания был выполнен запрос для получения последних четырех документов коллекции numbers.

3) Анализ плана выполнения запроса без индекса

```

Atlas atlas-12dqp-shard-0 [primary] learn> db.numbers.explain("executionStats").find()
|
|   .sort({value: -1})
|   .limit(4)
|
| {
|   explainVersion: '1',
|   queryPlanner: {
|     namespace: 'learn.numbers',
|     parsedQuery: {},
|     indexFilterSet: false,
|     queryHash: 'BA27D965',
|     planCacheShapeHash: 'BA27D965',
|     planCacheKey: '7A892B81',
|     optimizationTimeMillis: 0,
|     maxIndexedOrSolutionsReached: false,
|     maxIndexedAndSolutionsReached: false,
|     maxScansToExplodeReached: false,
|     prunedSimilarIndexes: false,
|     winningPlan: {
|       isCached: false,
|       stage: 'SORT',
|       sortPattern: { value: -1 },
|       memLimit: 33554432,
|       limitAmount: 4,
|       type: 'simple',
|       inputStage: { stage: 'COLLSCAN', direction: 'forward' }
|     },
|     rejectedPlans: []
|   },
|   executionStats: {
|     executionSuccess: true,
|     nReturned: 4,
|     executionTimeMillis: 128,
|     totalKeysExamined: 0,
|     totalDocsExamined: 102330,
|   }
| }

```

Рис. 58: Получение анализа запроса без индекса

В поле `executionTimeMillis` значение 128, значит время выполнения запроса составило: 128ms.

4) Создание индекса по полю `value`

```

[Atlas atlas-12dqp-shard-0 [primary] learn> db.numbers.createIndex({value: 1})
value_1

```

Рис. 59: Создание индекса по полю `value`

5) Получение информации об индексах

```

[Atlas atlas-12dqp-shard-0 [primary] learn> db.numbers.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]

```

Рис. 60: Получение списка индексов

6) Повторное выполнение запроса

```

Atlas atlas-12dqp-shard-0 [primary] learn> db.numbers.find()
|
|   .sort({value: -1})
|   .limit(4)
|
| {
|   _id: ObjectId('6a185f25bf50771e4023c661'), value: 99999 },
|   _id: ObjectId('6a185f25bf50771e4023c660'), value: 99998 },
|   _id: ObjectId('6a185f25bf50771e4023c65f'), value: 99997 },
|   _id: ObjectId('6a185f25bf50771e4023c65e'), value: 99996 }
| }

```

Рис. 61: Повторное выполнение запроса

7) Анализ плана запроса с индексом

```
Atlas atlas-12dqp-d-shard-0 [primary] learn> db.numbers.explain("executionStats").find()
|
|   .sort({value: -1})
|   .limit(4)
|
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: 'BA27D965',
    planCacheShapeHash: 'BA27D965',
    planCacheKey: '7A892B81',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExploreReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'LIMIT',
      limitAmount: 4,
      inputStage: {
        stage: 'FETCH',
        inputStage: {
          stage: 'IXSCAN',
          keyPattern: { value: 1 },
          indexName: 'value_1',
          isMultiKey: false,
          multiKeyPaths: { value: [] },
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'backward',
          indexBounds: { value: [ '[MaxKey, MinKey]' ] }
        }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 0,
    totalKeysExamined: 4,
  }
}
```

Рис. 62: Получение анализа запроса с индексом

Теперь в поле `executionTimeMillis` значение 0, значит время выполнения запроса составило: 0ms за счет работы индекса.

Более эффективным является запрос с использованием индекса, так как MongoDB выполняет поиск по индексированной структуре данных без полного просмотра всей коллекции.